Instrukcja laboratoryjna: sieć Hyperledger Fabric w chmurze Azure

Technologie chmurowe, Zadanie 5: Blockchain w chmurze

1. Cel i wymagania

Uruchamiamy sieć permissioned Hyperledger Fabric dla konsorcjum czterech firm logistycznych na maszynie wirtualnej w **Microsoft Azure**: organizacje Org1–Org4, każda z własnym węzłem (peer) i kopią księgi, orderer (konsensus Raft) i kanał mychannel. Wdrażamy smart kontrakt (chaincode) asset-transfer-basic i wykonujemy transakcje zapisu, odczytu i aktualizacji przesyłki. Org3 dodajemy oficjalnym skryptem addOrg3, a Org4 jego kopią. Domyślna polityka endorsementu (MAJORITY) wymaga podpisów 3 z 4 organizacji. Bez kryptowalut i bez danych osobowych.

Środowisko: subskrypcja Azure for Students (bez karty płatniczej, kredyt studencki), VM Standard_B2s_v2 (2 vCPU, 8 GiB RAM), Ubuntu Server 24.04 LTS, region Sweden Central, Windows z PowerShellem (klient SSH); Hyperledger Fabric 2.5.16, Fabric CA 1.5.17. Koszt VM: 0,0864 USD/h z kredytu (źródło: cena w kreatorze VM w portalu Azure, data sprawdzenia: 25.10.2026).

2. Przygotowanie konta i kontrola kosztów

- **Grupa zasobów** `fabric-demo` (Grupy zasobów → Utwórz) z tagiem `projekt=chmura2-grupa5`. Wszystkie zasoby trafiają do tej grupy, więc na końcu usuwamy ją jednym kliknięciem.
- **Budżet** `budzet-projekt`: 2 € miesięcznie, alerty e-mail przy 50% i 100% (Subskrypcje → Azure for Students → Zarządzanie kosztami → Budżety).
- Nie podajemy karty i nie przechodzimy na Pay-As-You-Go: po wyczerpaniu kredytu Azure wyłącza subskrypcję, więc nie ma ryzyka opłat.

3. Utworzenie maszyny wirtualnej

Maszyny wirtualne → Utwórz → Maszyna wirtualna, następnie zakładki kreatora:

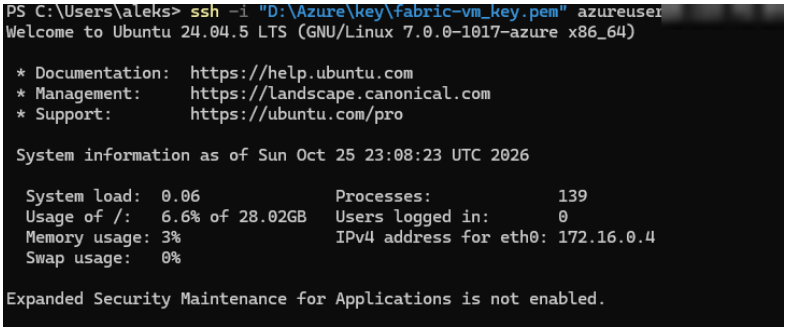
Ustawienie w kreatorze VM	Wartość
Grupa zasobów / nazwa	fabric-demo / fabric-vm
Region / dostępność	Sweden Central / bez nadmiarowości infrastruktury
Obraz / rozmiar	Ubuntu Server 24.04 LTS x64 Gen2 / Standard_B2s_v2
Uwierzytelnianie	klucz SSH RSA, użytkownik azureuser, nowa para fabric-vm_key
Publiczne porty przychodzące	Brak (cały ruch z Internetu zablokowany)
Dysk systemowy	30 GiB Standard SSD, usuwany razem z VM
Sieć	NSG podstawowa, publiczny IP i karta usuwane z VM, bez load balancera
Zarządzanie	automatyczne zamykanie 01:30 z powiadomieniem e-mail
Wyłączone	Spot, kopie zapasowe, monitoring płatny, SQL Server, rezerwacja
Tag	projekt = chmura2-grupa5 (wszystkie zasoby)

W subskrypcji studenckiej większość rodzin VM ma limit **0 vCPU** (błąd QuotaExceeded przy weryfikacji, np. D2s_v5). Dostępne rodziny sprawdzamy w Subskrypcja → Użycie i przydziały (dostawca Compute, region): rodzina **Bsv2** ma limit 10 vCPU. Weryfikacja kończy się przyciskiem Utwórz; klucz `fabric-vm_key.pem` pobieramy **tylko raz** do folderu poza repozytorium (np. `D:\Azure\key`).

4. Dostęp SSH tylko z naszego adresu IP

VM → Sieć → Utwórz regułę portu przychodzącego: Źródło **Mój adres IP**, usługa SSH (22/TCP), Zezwalaj, priorytet 100, nazwa `SSH-moj-IP`. Pozostały ruch blokuje domyślna reguła `DenyAllInBound`. Na Windowsie klucz musi być dostępny tylko dla właściciela (inaczej błąd UNPROTECTED PRIVATE KEY FILE); w PowerShellu:

```
icacls "D:\Azure\key\fabric-vm_key.pem" /inheritance:r
icacls "D:\Azure\key\fabric-vm_key.pem" /grant:r "${env:USERNAME}:R"
icacls "D:\Azure\key\fabric-vm_key.pem" /remove *S-1-5-11 *S-1-5-32-545
ssh -i "D:\Azure\key\fabric-vm_key.pem" azureuser@<IP_VM>
```



Rys. 1. Logowanie przez SSH do VM fabric-vm w Azure (adres IP zamazany)

5. Instalacja narzędzi, Dockera i Hyperledger Fabric

Na VM (`azureuser@fabric-vm`) instalujemy git, curl, jq, Go i Dockera oraz dodajemy użytkownika do grupy docker. Zamiast restartu wystarczy `exit` i ponowne połączenie SSH; potem `groups` musi zawierać `docker`, a `docker run hello-world` wypisuje „Hello from Docker!”. Następnie oficjalny skrypt pobiera fabric-samples, obrazy Dockera i programy (`peer`, `configtxgen`) do `fabric-samples/bin`:

```
sudo apt update && sudo apt install -y git curl jq golang-go docker.io docker-compose-v2
sudo usermod -aG docker $USER
exit                # ponowne ssh, potem: groups; docker run hello-world
mkdir -p ~/fabric && cd ~/fabric
curl -sSLO \
https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh --fabric-version 2.5.16 --ca-version 1.5.17 docker samples binary
~/fabric/fabric-samples/bin/peer version          # Version: v2.5.16
docker images | grep fabric-peer                  # tagi 2.5.16 i latest
```

Jeśli brak tagu `latest` (używa go skrypt `addOrg3`):

```
docker tag hyperledger/fabric-peer:2.5.16 hyperledger/fabric-peer:latest
```

6. Sieć z dwiema organizacjami i wdrożenie chaincode'u

```
cd ~/fabric/fabric-samples/test-network
./network.sh down
./network.sh up createChannel
./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-go -ccl go
```

`down` czyści pozostałości (komunikaty „no such volume” są normalne). `up createChannel` tworzy Org1, Org2, orderer i kanał `mychannel` („Channel 'mychannel' joined”). `deployCC` (ok. 2 min) instaluje i zatwierdza chaincode w Org1 i Org2; wynik kończy się linią `Approvals: [Org1MSP: true, Org2MSP: true]`.

7. Przygotowanie plików dla Org4 (jednorazowo)

Kopiujemy skrypt Org3 jako `addOrg4`, zamieniając nazwy (org3 → org4) i porty (1105x → 1305x):

```
cp -r addOrg3 addOrg4
cp -r scripts/org3-scripts scripts/org4-scripts
grep -rIl -e org3 -e Org3 -e ORG3 -e '1105[0-9]' addOrg4 scripts/org4-scripts | \
  xargs sed -i -e 's/org3/org4/g' -e 's/Org3/Org4/g' -e 's/ORG3/ORG4/g' \
  -e 's/1105\([0-9]\)/1305\1/g' -e 's/^ORG=3$/ORG=4/' \
  -e 's/setGlobals 3$/setGlobals 4/' -e 's/joinChannel 3$/joinChannel 4/' \
  -e 's/setAnchorPeer 3$/setAnchorPeer 4/'
find addOrg4 -depth -name '*rg3*' | while read p; do \
  mv "$p" "$(dirname "$p")/$(basename "$p" | sed 's/org3/org4/g;s/Org3/Org4/g')"; done
chmod +x addOrg4/*.sh scripts/org4-scripts/*.sh
grep -rIn -i org3 addOrg4 scripts/org4-scripts     # nie wypisuje nic
```

Następnie dopisujemy Org4 do `scripts/envVar.sh` (dane administratora) i `scripts/setAnchorPeer.sh` (adres anchor peera); skrypt wypisuje `OK`:

```
python3 - <<'PY'
import re
p = "scripts/envVar.sh"; s = open(p).read()
l3 = [l for l in s.splitlines() if l.startswith("export PEER0_ORG3_CA=")][0]
s = s.replace(l3, l3 + "\n" + l3.replace("ORG3", "ORG4").replace("org3", "org4"), 1)
m = re.search(r'\n([ \t]*)else\n[ \t]*errorln "ORG Unknown"', s); assert m
i = m.group(1); j = i + "  "
new = ("\n" + i + "elif [ $USING_ORG -eq 4 ]; then"
  "\n" + j + "export CORE_PEER_LOCALMSPID=Org4MSP"
  "\n" + j + "export CORE_PEER_TLS_ROOTCERT_FILE=$PEER0_ORG4_CA"
  "\n" + j + "export CORE_PEER_MSPCONFIGPATH=${TEST_NETWORK_HOME}/organizations/"
  "peerOrganizations/org4.example.com/users/Admin@org4.example.com/msp"
  "\n" + j + "export CORE_PEER_ADDRESS=localhost:13051")
s = s[:m.start()] + new + s[m.start():]; open(p, "w").write(s)
p = "scripts/setAnchorPeer.sh"; s = open(p).read()
m = re.search(r'\n([ \t]*)else\n[ \t]*errorln "Org\$\{ORG\} unknown"', s); assert m
i = m.group(1); j = i + "  "
new = ("\n" + i + "elif [ $ORG -eq 4 ]; then" + "\n" + j + 'HOST="peer0.org4.example.com"'
  + "\n" + j + "PORT=13051")
s = s[:m.start()] + new + s[m.start():]; open(p, "w").write(s); print("OK")
PY
grep -n "ORG4\|eq 4" scripts/envVar.sh scripts/setAnchorPeer.sh   # 4 linie
```

Uwaga: sekcję 7 wykonujemy tylko raz; pliki zostają na dysku VM także po jej zatrzymaniu.

8. Dodanie Org3 i Org4 oraz instalacja chaincode'u

Każdy skrypt (ok. 1 min) generuje certyfikaty, uruchamia peer organizacji i dołącza go do kanału:

```
cd addOrg3 && ./addOrg3.sh up && cd ..
cd addOrg4 && ./addOrg4.sh up && cd ..
docker ps --format 'table {{.Names}}\t{{.Status}}'
```

```
azureuser@fabric-vm:~/fabric/fabric-samples/test-network$ docker ps --format 'table {{.Names}}\t{{.Status}}'
NAMES                                                                                                STATUS
peer0.org4.example.com                                                                               Up 50 seconds
peer0.org3.example.com                                                                               Up 2 minutes
dev-peer0.org2.example.com-basic_1.0-d0b27e1af09f3039dabac82678c64e8807311f9405008efb867ba8895dd643aa   Up 27 minutes
dev-peer0.org1.example.com-basic_1.0-d0b27e1af09f3039dabac82678c64e8807311f9405008efb867ba8895dd643aa   Up 27 minutes
orderer.example.com                                                                                  Up 30 minutes
peer0.org2.example.com                                                                               Up 30 minutes
peer0.org1.example.com                                                                               Up 30 minutes
azureuser@fabric-vm:~/fabric/fabric-samples/test-network$
```

Rys. 2. Sieć w Azure: cztery peery (Org1–Org4) i orderer działają

Org3 i Org4 instalują chaincode i zatwierdzają jego definicję (bez ponownego commit), funkcjami ze skryptów `test-network` (ok. 3–5 min):

```
export PATH=${PWD}/../bin:$PATH
export FABRIC_CFG_PATH=$PWD/../config/
export CHANNEL_NAME=mychannel CC_NAME=basic CC_VERSION=1.0 CC_SEQUENCE=1
export INIT_REQUIRED="" CC_END_POLICY="" CC_COLL_CONFIG=""
export DELAY=3 MAX_RETRY=5 VERBOSE=false
. scripts/utils.sh
. scripts/envVar.sh
. scripts/ccutils.sh
PACKAGE_ID=$(peer lifecycle chaincode calculatepackageid basic.tar.gz)
for ORG in 3 4; do installChaincode $ORG; queryInstalled $ORG; approveForMyOrg $ORG; done
setGlobals 1
peer lifecycle chaincode querycommitted -C mychannel -n basic
```

```
+ res=0
2026-10-26 00:05:54.673 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [fea55f4bb91e3d482d13e0b616217e72cd632739446 52b6
d54f9f0dda64162cb] committed with status (VALID) at localhost:13051
Chaincode definition approved on peer0.org4 on channel 'mychannel'
Using organization 1
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true,
Org3MSP: true, Org4MSP: true]
```

Rys. 3. Definicja chaincode'u zatwierdzona przez cztery organizacje

9. Transakcje: zapis, odczyt, aktualizacja

Otwieramy drugie okno PowerShella i drugie połączenie SSH (nowy terminal) i ustawiamy środowisko administratora Org1. Zmienna `INVOKE` zbiera podpisy peerów Org1, Org2 i Org3 (3 z 4), a `--waitForEvent` czeka na zapis w bloku. Zmienne działają tylko w tym terminalu. Znak `\` na końcu linii oznacza, że polecenie ciągnie się w następnej linii; kopiujemy cały blok.

```
cd ~/fabric/fabric-samples/test-network
export PATH=${PWD}/../bin:$PATH
export FABRIC_CFG_PATH=$PWD/../config/
export CORE_PEER_TLS_ENABLED=true
ORDERER_CA=${PWD}/organizations/ordererOrganizations/example.com/orderers/\
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
ORG1_CA=${PWD}/organizations/peerOrganizations/org1.example.com/\
peers/peer0.org1.example.com/tls/ca.crt
ORG2_CA=${PWD}/organizations/peerOrganizations/org2.example.com/\
peers/peer0.org2.example.com/tls/ca.crt
ORG3_CA=${PWD}/organizations/peerOrganizations/org3.example.com/\
peers/peer0.org3.example.com/tls/ca.crt
ORG4_CA=${PWD}/organizations/peerOrganizations/org4.example.com/\
peers/peer0.org4.example.com/tls/ca.crt
export CORE_PEER_LOCALMSPID=Org1MSP
export CORE_PEER_TLS_ROOTCERT_FILE=$ORG1_CA
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/\
org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
INVOKE="peer chaincode invoke -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA \
  -C mychannel -n basic --waitForEvent \
  --peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_CA \
  --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_CA \
  --peerAddresses localhost:11051 --tlsRootCertFiles $ORG3_CA"
```

Pola kontraktu = dane przesyłki: **ID** – numer, **Color** – status, **Size** – waga (kg), **Owner** – firma, **AppraisedValue** – wartość (bez danych osobowych). `invoke` zmienia stan przez orderera, `query` tylko czyta.

```
$INVOKE -c '{"function":"CreateAsset","Args":["PRZESYLKA-001","nadana","12","FirmaA","500"]}'
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","PRZESYLKA-001"]}' | jq
$INVOKE -c '{"function":"TransferAsset","Args":["PRZESYLKA-001","FirmaB"]}'
$INVOKE -c '{"function":"UpdateAsset",
"Args":["PRZESYLKA-001","w_transporcie","12","FirmaB","500"]}'
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","PRZESYLKA-001"]}' | jq
```

Każda transakcja ma status `(VALID)` na trzech peerach i `status:200`; `TransferAsset` zwraca poprzedniego właściciela (`payload:"FirmaA"`), a odczyt pokazuje `w_transporcie` i `FirmaB`. **Odczyt z węzła Org4:**

```
export CORE_PEER_LOCALMSPID=Org4MSP
export CORE_PEER_TLS_ROOTCERT_FILE=$ORG4_CA
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/\
org4.example.com/users/Admin@org4.example.com/msp
export CORE_PEER_ADDRESS=localhost:13051
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","PRZESYLKA-001"]}' | jq
```

```
azureuser@fabric-vm:~/fabric/fabric-samples/test-network$ export CORE_PEER_LOCALMSPID=Org4MSP
export CORE_PEER_TLS_ROOTCERT_FILE=$ORG4_CA
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org4.example.com/users/Admin@org4.example.com/msp
export CORE_PEER_ADDRESS=localhost:13051
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","PRZESYLKA-001"]}' | jq
{
  "AppraisedValue": 500,
  "Color": "w_transporcie",
  "ID": "PRZESYLKA-001",
  "Owner": "FirmaB",
  "Size": 12
}
```

Rys. 4. Odczyt z peera Org4 (nie podpisywał transakcji): stan identyczny jak w Org1

Skrypt `demo_4org.sh`**.** Ten sam scenariusz (zapis, przekazanie, zmiana statusu i odczyt z Org2, Org3 i Org4), z pauzą po każdym kroku, wykonuje skrypt `demo_4org.sh` z repozytorium projektu; używamy go na pokazie na żywo. Skrypt sam ustawia zmienne i sprawdza, czy sieć i chaincode działają. Kopiujemy go na VM (PowerShell, w folderze ze skryptem) i uruchamiamy po sekcjach 6 i 8:

```
scp -i "D:\Azure\key\fabric-vm_key.pem" demo_4org.sh azureuser@<IP_VM>:~/
```

```
cd ~/fabric/fabric-samples/test-network
mv ~/demo_4org.sh .
sed -i 's/\r$//' demo_4org.sh      # usuwa windowsowe końce linii
chmod +x demo_4org.sh
./demo_4org.sh                     # gdy PRZESYLKA-001 już istnieje: ./demo_4org.sh PRZESYLKA-002
```

10. Po pracy: sprzątanie, zatrzymanie VM i usunięcie zasobów

Po każdej sesji usuwamy kontenery, wolumeny i obrazy chaincode'u; wolumen Org4 osobno, bo `network.sh down` zna tylko Org1–Org3:

```
cd ~/fabric/fabric-samples/test-network
./network.sh down
docker volume rm compose_peer0.org4.example.com
docker container prune -f
docker ps -a           # pusta lista
docker volume ls       # bez wolumenów compose_...
exit
```

- Portal: `fabric-vm` → **Zatrzymaj**. Stan „Zatrzymano (bez alokacji)” oznacza brak opłat za obliczenia; naliczany jest tylko dysk i adres IP (ok. 0,2 USD dziennie; źródło: szacunek na podstawie zużycia kredytu w portalu Azure, data sprawdzenia: 26.10.2026). Nie wyłączamy VM przez `sudo shutdown`, bo pozostaje przydzielona i nadal płatna.
- Wznowienie: Uruchom, SSH, sekcje 6, 8 i 9 (pliki z sekcji 7 zostają, ok. 15 min). Przy innym adresie IP poprawiamy źródło w regule `SSH-moj-IP`.
- Usunięcie zasobów po ćwiczeniu: Grupy zasobów → `fabric-demo` → Usuń grupę zasobów → wpisujemy nazwę grupy i potwierdzamy. Usunięcie jest nieodwracalne i obejmuje VM, dysk, publiczny IP i regułę sieciową. Jeśli Azure automatycznie utworzył grupę `NetworkWatcherRG`, usuwamy ją tak samo. Na koniec sprawdzamy listę grup zasobów; po kilku minutach koszt przestaje być naliczany.